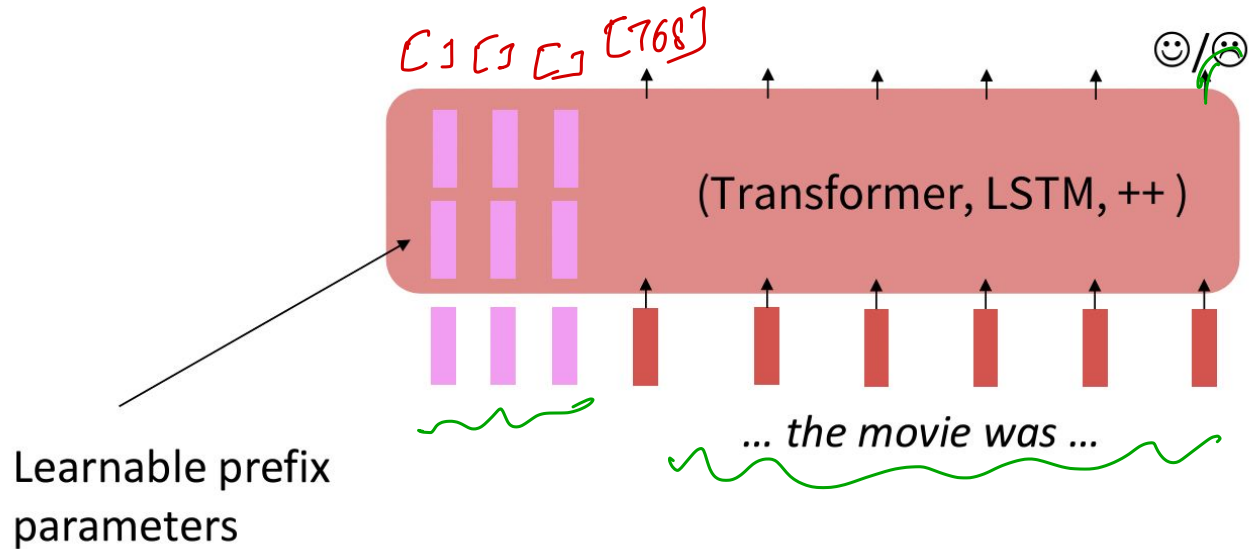


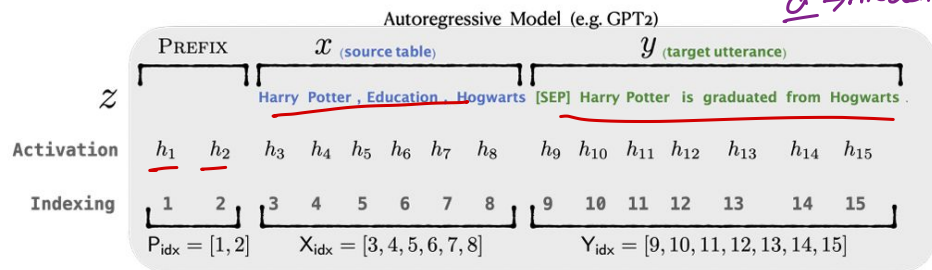
An input perspective: Prefix-tuning

Prefix-tuning adds a prefix of parameters, and freezes all pretrained parameters.



How does it work?

size P prefix $L \rightarrow$ layers
 $\bar{d} \rightarrow$ model - dim



Summarization Example

Article: Scientists at University College London discovered people tend to think that their hands are wider and their fingers are shorter than they truly are. They say the confusion may lie in the way the brain receives information from different parts of the body. Distorted perception may dominate in some people, leading to body image problems ... [ignoring 308 words] could be very motivating for people with eating disorders to know that there was a biological explanation for their experiences, rather than feeling it was their fault."

Summary: The brain naturally distorts body image - a finding which could explain eating disorders like anorexia, say experts.

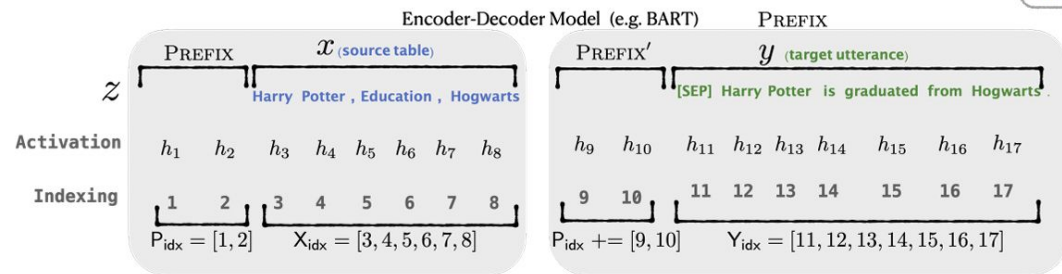


Table-to-text Example

Table: name[Clowns] customer-rating[1 out of 5] eatType[coffee shop] food[Chinese] area[riverside] near[Clare Hall]

Textual Description: Clowns is a coffee shop in the riverside area near Clare Hall that has a rating 1 out of 5 . They serve Chinese food .

Figure 2: An annotated example of prefix-tuning using an autoregressive LM (top) and an encoder-decoder model (bottom). The prefix activations $\forall i \in P_{idx}, h_i$ are drawn from a trainable matrix P_θ . The remaining activations are computed by the Transformer.

Which layers do we tune?

Prefix-tuning only the embeddings

- When only at the embedding layer, trainable parameters are $d_{model} \times |P_{idx}|$.
- In subsequent layers, operations are like usual transformer.

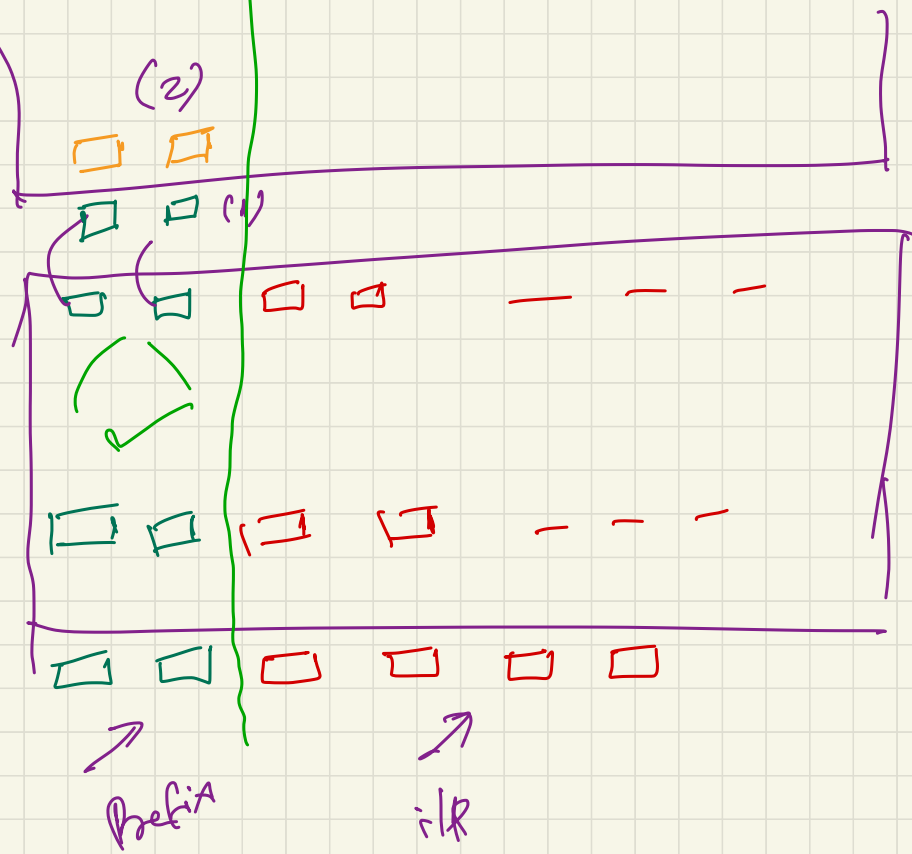
Prefix-tuning all the layers

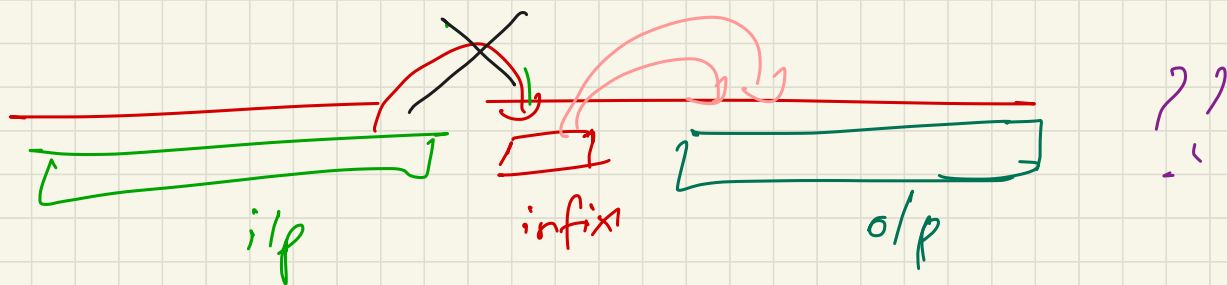
- When applied at all layers, trainable parameters are $d_{model} \times |P_{idx}| \times L$.
- In subsequent layers, operations are like usual transformer for all other tokens. For prefix tokens, activations are taken directly from a trainable parameter set.



1. Addⁿ parameters
only at embedd

2. Addⁿ parameters
at all layers





prefix

i/p

o/p

Can we also do infix?

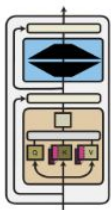
- $z = \{PREFIX; x; y\}$ vs. $z = \{x; INFIX; y\}$
- For decoder-only models, while prefixing can affect the activations of both x and y , infixing only impacts the activations of y
- In general, infixing slightly underperforms prefix, but both can also be used together.

How is the performance?

	E2E					WebNLG								
	BLEU	NIST	MET	R-L	CIDEr	BLEU			MET			TER ↓		
						S	U	A	S	U	A	S	U	A
GPT-2 _{MEDIUM}														
FT-FULL	68.8	8.71	46.1	71.1	2.43	64.7	26.7	45.7	0.46	0.30	0.38	0.33	0.78	0.54
FT-TOP2	68.1	8.59	46.0	70.8	2.41	53.6	18.9	36.0	0.38	0.23	0.31	0.49	0.99	0.72
ADAPTER(3%)	68.9	8.71	46.1	71.3	2.47	60.5	47.9	54.8	0.43	0.38	0.41	0.35	0.46	0.39
ADAPTER(0.1%)	66.3	8.41	45.0	69.8	2.40	54.5	45.1	50.2	0.39	0.36	0.38	0.40	0.46	0.43
PREFIX(0.1%)	70.3	8.82	46.3	72.1	2.46	62.9	45.3	55.0	0.44	0.37	0.41	0.35	0.51	0.42
GPT-2 _{LARGE}														
FT-FULL	68.5	8.78	46.0	69.9	2.45	65.3	43.1	55.5	0.46	0.38	0.42	0.33	0.53	0.42
Prefix	70.3	8.85	46.2	71.7	2.47	63.4	47.7	56.3	0.45	0.39	0.42	0.34	0.48	0.40
SOTA	68.6	8.70	45.3	70.8	2.37	63.9	52.8	57.1	0.46	0.41	0.44	-	-	-



Input
composition



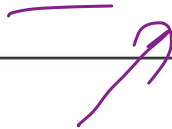
Parameter
efficiency

Only add a
small number
of parameters



Training
efficiency

Extend the model's context
window



Inference
efficiency

Performance

Requires
large models
to perform
well

Parameter perspective

- Assume we have a pre-trained autoregressive language model $P_\phi(y|x)$
 - E.g., GPT based on Transformer
- Adapt this pretrained model to downstream tasks (e.g., summarization, NL2SQL, reading comprehension)
 - Training dataset of context-target pairs $\{(x_i, y_i)\}_{i=1, \dots, N}$
- During full fine-tuning, we update ϕ_o to $\phi_o + \Delta\phi$ by following the gradient to maximize the conditional language modeling objective

$$\max_{\phi} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_\phi(y_t|x, y_{<t}))$$

Parameter perspective

- For each downstream task, we learn a different set of parameters $\Delta\phi$
 - $|\Delta\phi| = |\phi_o|$
 - GPT-3 has a $|\phi_o|$ of 175 billion
 - Expensive and challenging for storing and deploying many independent instances
- **Key idea:** encode the task-specific parameter increment $\Delta\phi = \Delta\phi(\Theta)$ by a smaller-sized set of parameters Θ , $|\Theta| \ll |\phi_o|$
- The task of finding $\Delta\phi$ becomes optimizing over Θ

$$\max_{\Theta} \sum_{(x,y)} \sum_{t=1}^{|y|} \log(P_{\phi_o + \Delta\phi(\Theta)}(y_t | x, y_{<t}))$$

$$\frac{\Delta \phi}{d^2}$$

$$A \times B \int_{dx} dx$$

$$d^2$$

$\approx$

$$M_1 \int_{dx} dx \quad M_2^T \int_{dx} dx$$

$$2 \times dx \times 2$$

$$2 < \frac{d}{2}$$

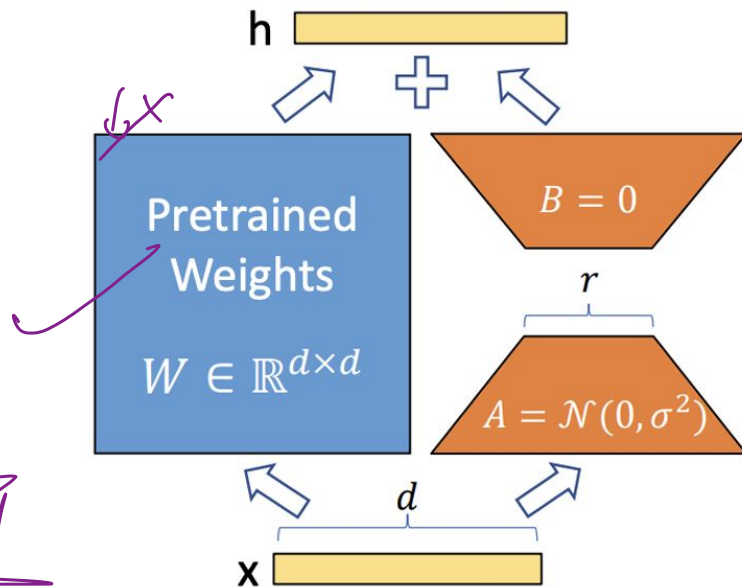
Low-rank-parameterized update matrices

- Updates to the weights have a low “intrinsic rank” during adaptation (Aghajanyan et al. 2020)
- $W_0 \in \mathbb{R}^{d \times k}$: a pretrained weight matrix
- Constrain its update with a low-rank decomposition:

$$W_0 + \Delta W = W_0 + BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$

- Only A and B contain **trainable** parameters



LoRA: What happens during training and inference?

- $\underline{W_0} + \underline{\Delta W} = \underline{W_0} + \underline{BA}$
- During training, W_0 is frozen and does not receive gradient updates.
- For $h = \underline{W_0 x}$, forward pass would be

$$h = W_0 x + \Delta W x = \underline{W_0 x} + \underline{BAx}$$

At inference time?

Use the matrix $W = W_0 + BA$, so no additional latency

W_{FF}

W_q

W_k

W_v

W_o

Which weight matrices to apply to?

Which weight matrices in Transformers should we apply LoRA to?

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Adapting both W_q and W_v gives the best performance overall.

What is the optimal rank r for LoRA?

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

LoRA already performs competitively with a very small r

Understanding LoRA Parameters over GPT-3

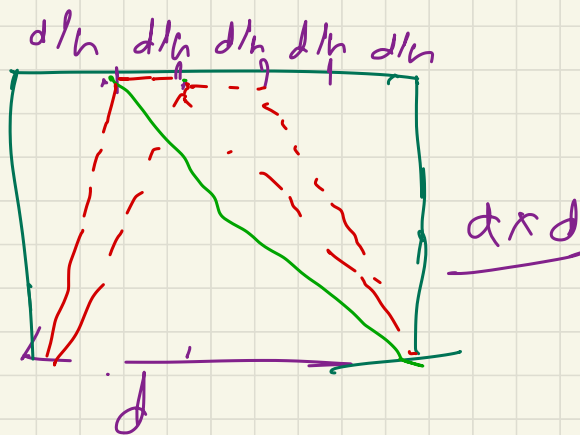
- For GPT-3, $d_{model} = 12288$, 96 decoders and 96 attention heads

LoRA

$r_v = 2$	4.7 M
$r_q = r_v = 1$	4.7 M
$r_q = r_v = 2$	9.4 M
$r_q = r_k = r_v = r_o = 1$	9.4 M
$r_q = r_v = 4$	18.8 M
$r_q = r_k = r_v = r_o = 2$	18.8 M
$r_q = r_v = 8$	37.7 M
$r_q = r_k = r_v = r_o = 4$	37.7 M
$r_q = r_v = 64$	301.9 M
$r_q = r_k = r_v = r_o = 64$	603.8 M

Layer L

w_0



≈ 9.4 million

$$2 \times 12288 \times 2 \times 96 \times 2$$

$$\boxed{2 \times d \times 2 \times L \times 2}$$

$$\cancel{[d \times 2 + \frac{d}{h} \times 2]} \times h \times L \times 2$$

Applying LoRA to Transformers

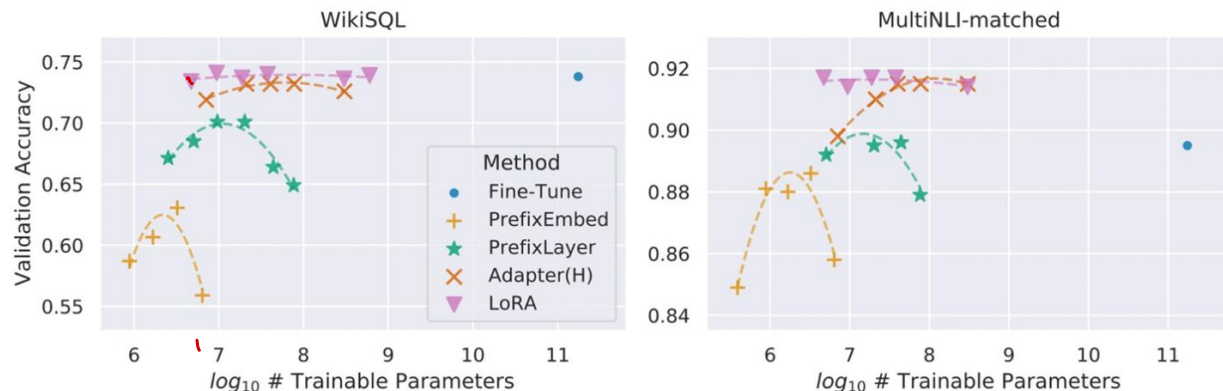
Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters

LoRA in Practice: Scaling to GPT-3

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

LoRA matches or exceeds the fine-tuning baseline on all three datasets



LoRA exhibits better scalability and task performance

Method	Hyperparameters	# Trainable Parameters	WikiSQL	MNLI-m
Fine-Tune	-	175B	73.8	89.5
✓ PrefixEmbed	$l_p = 32, l_i = 8$	0.4 M	55.9	84.9
	$l_p = 64, l_i = 8$	0.9 M	58.7	88.1
	$l_p = 128, l_i = 8$	1.7 M	60.6	88.0
	$l_p = 256, l_i = 8$	3.2 M	63.1	88.6
	$l_p = 512, l_i = 8$	6.4 M	55.9	85.8
✓ PrefixLayer	$l_p = 2, l_i = 2$	5.1 M	68.5	89.2
	$l_p = 8, l_i = 0$	10.1 M	69.8	88.2
	$l_p = 8, l_i = 8$	20.2 M	70.1	89.5
	$l_p = 32, l_i = 4$	44.1 M	66.4	89.6
	$l_p = 64, l_i = 0$	76.1 M	64.9	87.9
✓ Adapter ^H	$r = 1$	7.1 M	71.9	89.8
	$r = 4$	21.2 M	73.2	91.0
	$r = 8$	40.1 M	73.2	91.5
	$r = 16$	77.9 M	73.2	91.5
	$r = 64$	304.4 M	72.6	91.5
✓ LoRA	$r_v = 2$	4.7 M	73.4	91.7
	$r_q = r_v = 1$	4.7 M	73.4	91.3
	$r_q = r_v = 2$	9.4 M	73.3	91.4
	$r_q = r_k = r_v = r_o = 1$	9.4 M	74.1	91.2
	$r_q = r_v = 4$	18.8 M	73.7	91.3
	$r_q = r_k = r_v = r_o = 2$	18.8 M	73.7	91.7
	$r_q = r_v = 8$	37.7 M	73.8	91.6
	$r_q = r_k = r_v = r_o = 4$	37.7 M	74.0	91.7
	$r_q = r_v = 64$	301.9 M	73.6	91.4
	$r_q = r_k = r_v = r_o = 64$	603.8 M	73.9	91.4

*Comparison of various
PEFT methods*

Try this problem

Suppose you are using GPT-3 for the query auto-completion task, where the user types a prefix, and the system tries to complete this. You want to personalize this by fine-tuning individually for 10,000 users of the system. You have the following options:

1. You can use prefix-tuning at all the layers with length of prefix as 4 and length of infix as 4
2. You can use LoRA for the query and key matrices at all the layers, with rank $r = 2$
3. You can use adapters (2 adapters per layer), with the projection dimension as 2

Compare the number of parameters required for each of these options.

from table

Some variations of LoRA

KronA: Parameter Efficient Tuning with Kronecker Adapter

(low-rank decomposition suffers from limited representation power, KronA uses a different method, which is not low-rank)

VeRA: Vector-based Random Matrix Adaptation

(Significantly reduces the number of trainable parameters compared to LoRA, yet maintains the same performance)

8B parameters

1. Weights
2. Gradients
3. Momentum
4. Squared gradients
(32-bit)

} ~128GB

Efficient Fine-tuning for Quantized LMs